

Name: Gaigi P. Sukhatankar
Roll No: B027

SAPID: 60019220125
Branch: CSE(CB)

DE- ASSIGNMENT 1

Q-1. Explain in detail types of data Architecture - data warehouse, DWH characteristic, data lake and lambda architecture

⇒ (i) Data Warehouse

- Data warehouse (DWH) was proposed as a new type of database management system which would keep no transactional data but only summarized historical information for decision making purposes.
- DW's are modelled & structured differently, use different techniques for storage and retrieval and cater to a different set of users.
- Data warehouses are databases that store and maintain analytical data separately from transaction-oriented databases for the purpose of decision support.

Characteristics of DWH:

- Multidimensional conceptual view.
- Unlimited dimensions and aggregation levels.
- Unrestricted cross-dimensional operations.
- Dynamic sparse matrix handling.
- Client-server architecture.
- Multiuser support.
- Accessibility
- Transparency
- Intuitive data manipulation
- Inductive & deductive analysis.
- Flexible distributed reporting.

② Data Lake

A data lake is a centralized repository that allows you to store all your structured and unstructured data at any scale.

Primary components of data lake.

1) Metadata

→ Accountable for managing data structure.

→ can be used to boost computing efficiency & improve the usability.

2) Compute

→ Accountable for doing data crunching.

→ Computing engine can be combined with storage, as in a warehouse, or they might be entirely focused on computing.

3) Format

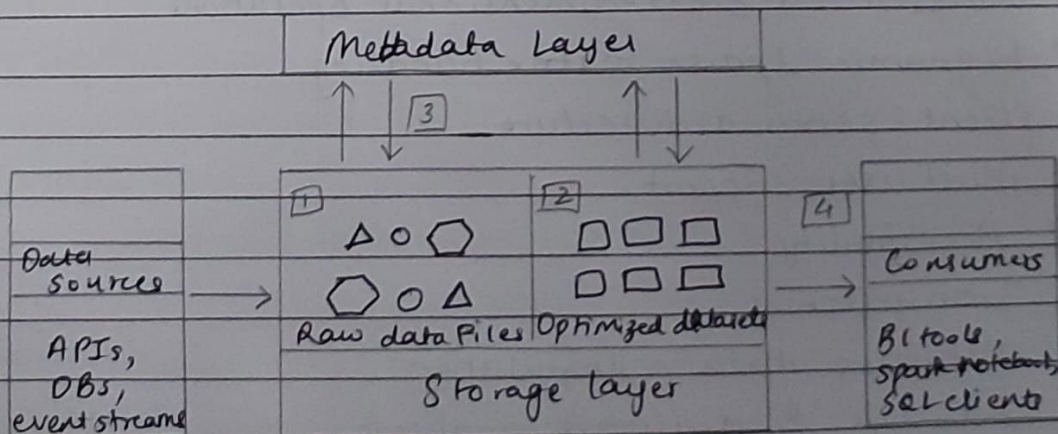
→ Data of any shape or format is stored at the storage layer.

→ Format has an impact on the available operations and the compatibility of compute engines.

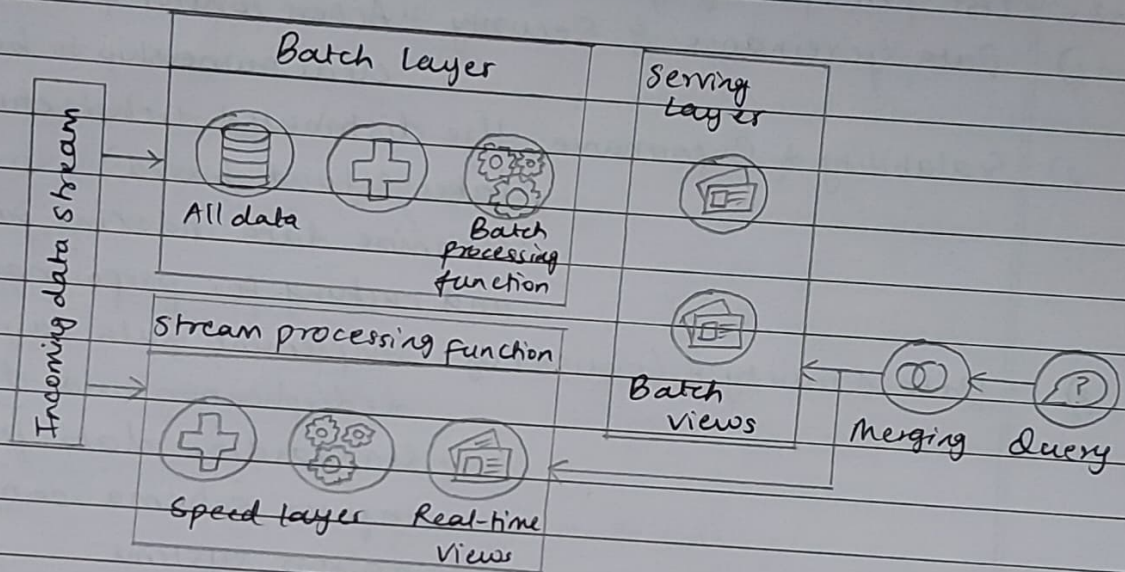
4) Storage

→ Where data is physically stored.

→ Durability, scalability and compatibility with other tools, engines and libraries are all requirements.



③ Lambda Architecture



Key Components

- Batch Layer: Saves incoming data as batch views.
- Speed Layer: Delivers data into batch layer.
- Serving Layer: Batch view and speed layer outcome traverse to serving layer.
- Lambda architecture provides a common architectural pattern for discussion.
- Provides a more clear picture of the complexities typically found in most organizations.
- Challenges in tackling lambda architecture
- Complete lambda architecture requires more than just a single system.
- Some challenges is delivering results to the business.
- Takes alot of knowledge/expertise/technology to tackle.
- Not typically a first step in Big Data implementation.

Q.2. List principles of Good Data Architecture.

- 1) Data Governance & Security : Access control & encryption,
• clear ownership to be established
- 2) Scalability & Performance : • Use distributed architecture like data lakes (cloud-based) & warehouses
• Optimize data indexing, partitioning and caching for performance.
- 3) Data Quality & Consistency : • Implement data validation, deduplication and cleaning.
• Standardize data forms & enforce schema consistency across systems.
• Use real-time monitoring to detect and resolve data quality issues.
- 4) Interoperability & Integration : • Use APIs, ETL, pipelines and data streaming
• Adopt open standards (JSON, Avro, Parquet) for data exchange.
• Enable cross-platform compatibility for multi-cloud and hybrid environments.
- 5) Flexibility and Modularity : • Design data models that support agility and future modifications.
• Use microservices and event-driven architectures to decouple dependencies.
• Allow easy schema evolution without breaking existing applications.

- 6) Real Time & Batch Processing:
 - Implement both ETL and ELT.
 - Use tools like Apache Kafka, Spark and Flink for real-time data ingestion.
- 7) Metadata Management & Lineage:
 - Maintain metadata catalogs.
 - Track data lineage.
 - Use automated tools.
- 8) Cost Optimization:
 - Optimize storage costs using tiered storage strategies.
 - Implement auto scaling.
 - Regularly audit data usage & limit it.

Q-3. Explain different steps of Extraction, Transformation & Loading.

- ⇒ ① Determine all the target data needed in the data warehouse: Define scope of data
- ② Determine all the data sources, both internal and external.
- ③ Prepare data mapping for Target Elements from sources: Map the fields from source systems to corresponding fields in the target schema
- ④ Establish comprehensive data extraction rules: Define the rules and mechanisms for extracting data from various sources. Includes specifying filters, frequency (eg batch/real-time) and any conditional logic necessary during extraction.
- ⑤ Determine data transformation and cleansing rules: Specify how raw data will be standardized, cleansed and transformed to meet quality and format requirements. Includes deduplication, handling nulls, converting datatypes and applying business rules.

- ⑥ Plan for aggregate tables: Design and plan pre-aggregated data set to support performance optimized reporting. These tables provide summarized insights and improve efficiency in analytical tools.
- ⑦ Organize data staging area and test tools: Set up a staging environment where raw data is initially loaded for validation, transformation and testing. Ensure proper version control, logging and test automation are in place to maintain data integrity.
- ⑧ Write procedures for all data loads: Develop ETL Scripts or procedures to automate data loading process. Include logging, exception handling and rollback mechanisms to make pipeline reliable, repeatable and auditable.
- ⑨ ETL for dimension tables: Load and transform data for dimension tables first as these are used to provide context (eg customer, product, region) to the measurable data in fact tables.
- ⑩ ETL for fact tables: Finally, load the fact tables containing transactional or measurable data (eg sales, revenue, clicks). These tables should reference the previously loaded dimensions and are the foundation for business metrics and KPIs.

Q-4. Apply data techniques and processes in data loading environment

- ⇒ ① Batch vs Real-Time Loading: Loads large volume of data at scheduled intervals, ideal for systems where near real-time access isn't critical.
- ② Real-time: Enables continuous data flow, suitable for dynamic, fast-moving datasets like IoT, social media or financial feeds.

② Incremental Loading:

- Only new updated records are loaded rather than full dataset.
- Minimize data redundancy, reduces system load and boosts performance.
- Techniques: Change Data Capture (CDC), Timestamp based filtering or using delta files.

③ Data Validation & Integrity checks:

- Ensure referential integrity.
- Verify data types, null constraints and unique values.
- Log discrepancies for debugging or reruns.

④ Parallel Processing:

- Loads using multi-threading or distributed frameworks like Apache Spark or Talend to handle large-scale datasets efficiently.

⑤ Error handling and Recovery:

- Implement error logging mechanisms to track failed inserts or data mismatches.

⑥ Partitioning & Indexing

- Partition large tables to improve query and load performance.
- Apply proper indexing after data load to speed up analytical queries without slowing down the loading process.

⑦ Load Sequencing

- Dimension tables must be loaded before fact tables to ensure referential integrity.

⑧ Data Archival

- Archive old or unused data to reduce warehouse size and increase performance.

⑨ Automation & Scheduling:

- Use ETL orchestration tools (e.g. Apache Airflow, Azure Data Factory)

to automate loading process

② Monitoring and Auditing:

- Monitor data pipeline health and performance.
- Maintain detailed logs of data volume, load time and failure points for audit and debugging.

8.5. Apply Data Integration with Redundancy and Correlation analysis. Consider an example.

→ Inside a healthcare system,

1) chr-patients: coming from the EHR system.

lab-results: coming from the LIS system.

Redundancy check and Removal.

Detecting duplicates

```
SELECT DISTINCT UPPER(patient-name), patient-id, dob
FROM chr-patients;
```

```
SELECT patient-name, COUNT(*)
```

```
FROM chr-patients
```

```
GROUP BY patient-name
```

```
HAVING COUNT(*) > 1;
```

2) checking if age can be determined from DOB.

```
SELECT patient-id, DOB, patient-age,
```

```
DATE_DIFF(CURRENT_DATE(), DATE(DOB), YEAR) AS computed-age,
```

```
Age = DATE_DIFF(CURRENT_DATE(), DATE(DOB), YEAR) AS Is-age,
```

```
Correct FROM chr-patients;
```

If Is-age-correct is false, age is redundant/inconsistent.

Correlation.

Correlating Cholesterol vs age.

SELECT

e.patient_id,
e.patient_name,
l.test_result AS cholesterol-101

FROM

ehr_patients as e

JOIN

lis_results as l ON e.patient_id = l.patient_id

WHERE

l.test_name = 'cholesterol';

6. Analyze data reduction strategies: Attribute subset, selection, regression and log linear models.

⇒ Attribute subset selection: It is a dimensionality reduction method.

• Select a minimum set of features such that the probability distribution of different classes given the values for those features is close as possible to the original distribution given the values of all features reduce no. of patterns in the patterns, easier to understand.

• Regression and log-linear Models:

→ Linear Regression: Data modelled to fit a straight line.

• often use least-square method to fit the line

→ Multiple Regression: Allows a response variable Y to be modelled as a linear function of multidimensional feature vector.

→ log-linear Model: Approximates discrete multidimensional